

Register-Resident Keccak and NTT Acceleration for ML-KEM and ML-DSA on a SIMT GPU

Anonymous Authors

Abstract—ML-KEM and ML-DSA alternate polynomial arithmetic with Keccak, but use different coefficient widths and NTT layer counts. On a SIMT processor, their state placement and lane communication determine how much of this parallelism reaches a complete cryptographic request. We present register-resident Keccak and NTT collectives for an RV32 GPU. Stage collectives implement column reduction, word permutation, and row logic across lane registers, while explicit low/high outputs preserve the processor’s single-destination writeback interface. A single NTT unit shares 16 multipliers and XOR lane-pair routing between both schemes, selecting modulus-specific reduction for each instruction. We integrate these paths into complete ML-KEM-768 and ML-DSA-65 requests and compare against cooperative software. On a common XRT RTL simulation build, stages accelerate complete requests by $1.462\times$ and $1.552\times$ over the same shuffle mapping at one and eight requests in the K-only reference build. With equal loop expansion, round fusion provides $3.291\times$ permutation throughput but only $1.033\times$ complete-request speedup in a separate timing-model control. The shared NTT unit reduces complete KEM and DSA cycles by 4.913% and 5.939% against matched register-shuffle software NTTs. Separately, mapping DSA pointwise work onto the same core reduces eight-request XRT makespan by 11.687% with no new RTL.

Index Terms—ML-KEM, ML-DSA, Keccak, NTT, SIMT, RISC-V

I. INTRODUCTION

Standardized ML-KEM and ML-DSA make key establishment and signatures processor-design problems [1], [2]. Both alternate modular polynomial arithmetic and SHA-3/SHAKE hashing. Both contain regular computation, but their parallelism has different shapes: an NTT layer exposes independent butterflies, whereas a Keccak round repeatedly exchanges information across a small, tightly coupled state. Accelerating one arithmetic operation therefore leaves a second question unresolved: how should a programmable processor hold and exchange that state?

An extensible SIMT GPU offers an attractive setting for this question. A warp can distribute one polynomial or one permutation over ordinary lane registers; multiple warps can process independent requests. This avoids building a separate state store for every accelerated primitive and keeps the remaining sampling, encoding, and control code on the same processor. Yet a software mapping still spends instructions on shuffles, reductions, and 64-bit operations. On RV32, each Keccak word spans two registers, making cross-word rotations and collective writeback particularly important.

There are several plausible hardware boundaries. Fine-grained arithmetic instructions leave communication in software. Stage collectives replace structured communication and

logic while preserving intermediate states in GPRs. Fusing an entire round removes additional issue and dependency costs. A pointer engine moves the state into a dedicated unit for all 24 rounds. These choices trade instruction count, pipeline storage, state transfers, and concurrency. A faster isolated primitive need not produce a faster complete request.

Prior work supplies important building blocks. Optimized RISC-V software provides strong scalar baselines [3]; ML-Cube demonstrates a 25-thread shuffle mapping for Keccak [4]; and a published patent application describes a 64-bit warp-collective Keccak round [5]. The design question addressed here is how to realize a stage-factored interface on an RV32 SIMT datapath and determine whether coarser operations justify their implementation cost after the NTT is optimized and the same hardware serves ML-DSA.

We build this interface in Vortex [6]. Each collective consumes ordinary source vectors and produces one destination vector. Stage operations expose Keccak’s fixed communication patterns without introducing persistent accelerator state. A shared NTT unit removes a competing bottleneck in both schemes. Our contributions are:

- A stage-factored Keccak interface with explicit RV32 low/high outputs, fixed cross-lane communication, and ordinary register dependency and writeback handling.
- A half-bank of 16 shared multipliers and lane-pair routing for the seven-layer 16-bit ML-KEM NTT and eight-layer 32-bit ML-DSA NTT, integrated through both libraries’ native hooks.
- A controlled evaluation separating software mapping, collective instructions, and round fusion, with complete-request checks for both schemes and routed costs for K-only and shared K/D cores.

The central finding is that stages capture most of the application benefit available from register collectives. Round fusion substantially improves the permutation, but its incremental request-level gain is small; the pointer engine becomes preferable at eight concurrent requests. These outcomes make granularity a workload-dependent design choice.

II. BACKGROUND AND MOTIVATION

A. Lattice workloads and communication patterns

ML-KEM operates on degree-256 polynomials modulo 3329 and uses SHA-3/SHAKE for hashing and pseudorandom generation [1], [7]. Our workload is an ML-KEM-768 round trip: key generation, encapsulation, and decapsulation. A request in the integrated implementation executes 140 Keccak

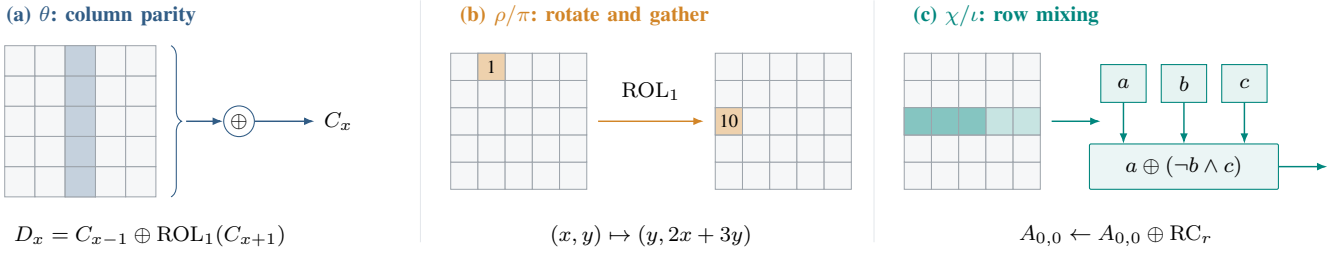


Fig. 1. Keccak’s fixed communication patterns. A state word maps to lane $x + 5y$; RV32 stores its low and high halves in two GPRs. The highlighted examples show a column reduction, the $A[1, 0]$ rotation and transfer to $B[0, 2]$, and a three-word row dependency. All coordinates wrap modulo five.

permutations, 15 forward NTTs, and nine inverse NTTs. Independent requests provide inter-warp parallelism; cooperative execution of each primitive provides intra-warp parallelism. ML-DSA-65 uses the same degree-256 polynomial length but 32-bit coefficients modulo 8,380,417 and a full eight-layer NTT [2]. Its round trip comprises key generation, signing, and verification; the signing rejection loop makes byte-exact checking necessary.

A Keccak-f[1600] state contains 25 words of 64 bits, arranged as $A[x, y]$ with $0 \leq x, y < 5$. Each of its 24 rounds applies column parity and correction (θ), rotation and word permutation ($\rho\pi$), and nonlinear row mixing plus a round constant ($\chi\iota$). Figure 1 illustrates these three communication patterns. Their connections are fixed by the algorithm, so a general sequence of shuffles is more flexible than the computation requires.

An ML-KEM NTT has seven layers of 128 butterflies. With coefficient $a[\ell + 32k]$ in register k of lane ℓ , the first three forward layers exchange registers within a lane. The remaining four exchange lanes at XOR distances 16, 8, 4, and 2. The inverse traverses these dependencies in reverse. ML-DSA adds distance 1 and uses 32-bit coefficients. This pairwise communication differs from Keccak’s reductions and permutations, motivating separate datapaths under a common register interface.

B. Why software mapping is not enough

A one-lane Keccak implementation serializes the state computation. Distributing the state across 25 lanes exposes parallel work, but the software must still form five-word parities, gather rotated words, and fetch row neighbors. In the final common-core experiment, this mapping alone reduces one-request cycles from 24.67 million for optimized RV32 assembly to 7.91 million. Hardware benefits must therefore be measured against the cooperative software, not attributed to the entire scalar-to-collective speedup.

Two constraints then shape the extension. First, an ordinary SIMT instruction writes one value per lane, while an RV32 Keccak stage produces two halves of a new state. Second, fusing more work reduces register-level communication but enlarges the operation and changes how independent warps share the execution resources. Our design addresses the first

constraint with explicit result halves and evaluates the second through matched stage, round, and pointer alternatives.

Related accelerator work reinforces the importance of the data mapping. NTT-PIM and BP-NTT arrange transform operations around local storage and arithmetic capabilities [8], [9]; CHAM evaluates specialized primitives within their surrounding cryptographic computation [10]. PacQ and SynGPU similarly connect SIMT data placement to datapath utilization [11], [12]. Here the state boundary is the ordinary lane register file. Compared with tightly coupled CPU extensions such as RISQ-V and scalar NTT instructions [13], [14], participating SIMT lanes supply distinct parts of one operation and receive their own results through normal vector writeback.

III. DESIGN

A. Execution model and core integration

Figure 2 shows the integration. The instruction decoder selects a collective execution unit within the ALU path. The existing operand interface supplies the source vectors; the result carries the warp and destination metadata through ordinary writeback. The scoreboard enforces register dependencies, and the unit observes the same backpressure as other execution units. Independent warps may interleave instructions without saving or restoring a private cryptographic context.

The implementation uses a 32-thread warp and 32 ALU lanes, so one issue presents the complete source vectors. Keccak assigns state to lanes 0–24 and requires all 32 lanes to execute with a full mask; padding lanes return zero. Loads and stores may be predicated, provided the warp reconverges before a collective. NTT butterflies permit pair-closed masks: each active lane’s partner must also be active. These contracts make the cross-lane operands explicit and avoid dependence on an absent or divergent producer.

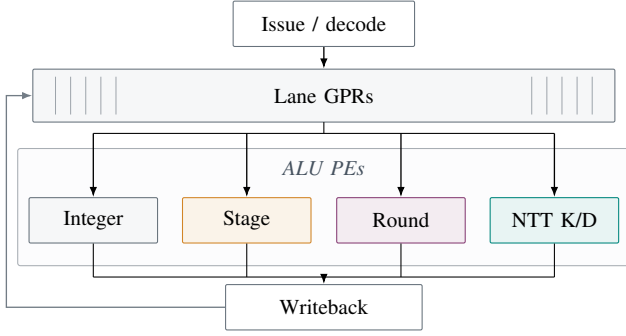
B. Stage-factored Keccak

The Stage design provides θ , $\rho\pi$, and $\chi\iota$ operations. Let $\text{ROL}_{64}(v, r)$ denote a 64-bit rotation and let all state indices be modulo five. The first stage computes

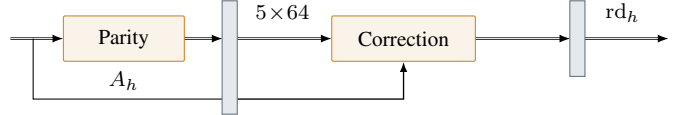
$$C[x] = \bigoplus_{y=0}^4 A[x, y], \quad (1)$$

$$T[x, y] = A[x, y] \oplus C[x - 1] \oplus \text{ROL}_{64}(C[x + 1], 1). \quad (2)$$

(a) Register interface

(b) Stage: θ

latency / II: 2 / 1



(c) Round

latency / II: 4 / 1

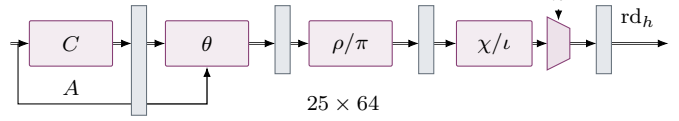


Fig. 2. Core integration. Stage, Round, and NTT units consume lane-register vectors and return one result per lane. The pointer backend instead accesses state through the LSU (not shown). Fixed collective routing is implemented in the new units. Vertical bars mark pipeline registers; h selects the output half. Latency/II refers to the unit interface, not the core pipeline.

Stage: 6 issues / round



Round: 2 issues / round

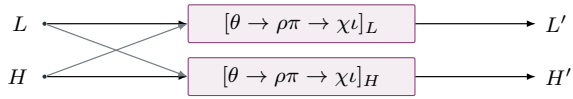


Fig. 3. Explicit RV32 result halves. Each node produces one destination vector; arrows denote data dependencies, not simultaneous issue. The θ and $\rho\pi$ low/high operations both read the same preceding pair of halves. Round fusion retains this interface while replacing the three stage pairs with one pair.

The datapath forms each parity once and distributes its correction to the five words in the column. The next stage rotates each word and routes it to a fixed destination:

$$B[y, (2x + 3y) \bmod 5] = \text{ROL}_{64}(T[x, y], r[x, y]). \quad (3)$$

Finally, row neighbors form

$$A'[x, y] = B[x, y] \oplus (\neg B[x + 1, y] \wedge B[x + 2, y]), \quad (4)$$

with the round constant XORed into $A'[0, 0]$. These fixed connections replace repeated software reductions and gathers.

a) Single-destination RV32 interface: Each stage has a low-half and a high-half instruction (Fig. 3). For θ and $\rho\pi$, both instructions read the same old low/high source vectors because rotations cross the 32-bit boundary. The low result must therefore use a temporary until the high instruction has consumed the old state. In contrast, each χ^L instruction reads only its corresponding state half and a uniform round index. The interface exposes this distinction to software and lets normal RAW dependencies enforce it.

A round requires six collective instructions and a full permutation 144. Persistent state occupies two GPRs per participating lane. Temporaries add to peak register use; the looped all-stage routine uses five GPRs for state, temporaries,

and loop control, with no stack frame or memory operation in the permutation loop. The θ datapath first registers the five column parities alongside the selected input half, then applies and registers the column correction. The $\rho\pi$ and χ^L paths maintain the same two-stage latency interface. All three accept one operation per cycle when unstalled. Carrying the instruction metadata with the data preserves warp identity and destination selection through backpressure. The two-cycle unit latency excludes operand collection, issue, and core writeback delay.

C. Granularity alternatives

a) Fused round: Round uses four registered steps: column parity, θ correction, $\rho\pi$, and χ^L (Fig. 2). The state and instruction metadata advance together through ready/valid pipeline buffers; different warps can occupy successive steps. This supports initiation interval one while placing the intermediate 1600-bit states inside the unit. On RV32, two instructions read the same input state and select the low or high output. Each invocation computes the internal full-word round; there is no hidden result shared between the two invocations. This preserves single-vector writeback and avoids a multi-destination transaction, at the cost of recomputing internal logic for the second result half. The round index is a literal, so the implementation expands all 24 rounds. The resulting 48 collective instructions per permutation remove two-thirds of Stage's issues and its intermediate architectural states. Our evaluation also expands Stage's rounds to separate fusion from loop control.

b) Pointer engine: The pointer alternative accepts the address of a 200-byte state, transfers it through an LSU client, performs all 24 rounds, and writes the state back. It amortizes issue overhead over a complete permutation, but places the state outside the GPR path. This boundary changes both the surrounding sponge operations and the resources shared by concurrent requests. We compare it within the same complete application rather than treating permutation latency alone as the selection criterion.

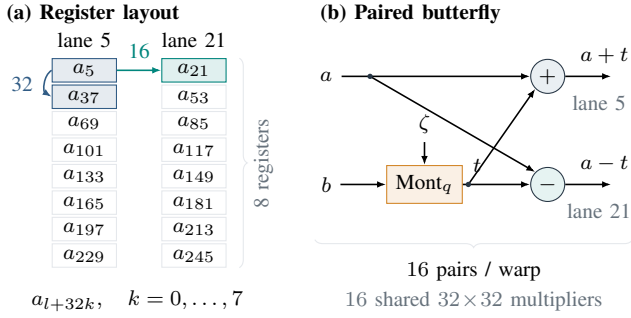


Fig. 4. Shared NTT mapping and a forward butterfly. Distance 32 connects registers within a lane, while distance 16 connects lanes 5 and 21. The pair-low lane supplies ζ ; one product feeds both results. The multiplier and routing bank is common to ML-KEM and ML-DSA; the selected reduction uses $q_K = 3329$ or $q_D = 8380417$.

c) *Integer width*: The RV64 version retains the same lane mapping but represents each word in one GPR. Stage then needs three instructions per round and Round needs one. This removes the explicit half selection without changing the algorithmic communication. It also widens the rest of the processor, so its benefit must be evaluated against full-core cost and complete requests, not just the halved collective count.

D. Shared NTT datapath and application mapping

Both schemes keep 256 coefficients in eight GPRs per lane. Figure 4 shows how butterfly distance selects either another register or another lane. Lane-local operations use Montgomery multiplication; cross-lane operations use paired butterfly collectives. For a forward Cooley–Tukey pair, the low lane supplies a and ζ , and the high lane supplies b :

$$t = \text{Mont}(b\zeta), \quad (a', b') = (a + t, a - t). \quad (5)$$

One product serves both lanes, and each lane receives only its own result. The inverse Gentleman–Sande form computes a sum and twiddle-scaled difference. ML-KEM reduces the 16-bit sum with Barrett arithmetic; ML-DSA retains the signed 32-bit sum. Each follows its library’s exact Montgomery convention and twiddle order.

A bank of 16 signed 32×32 variable-product multipliers serves 16 lane pairs per issue. An instruction mode bit selects a 16-bit or 32-bit Montgomery reduction; pair routing, product pipeline, and writeback are shared, while the constant-arithmetic reduction networks differ. ML-KEM inverse butterflies use a second beat for sum-times-20159 Barrett reduction, while ML-DSA inverse butterflies use one beat. Scalar products use two beats for lanes 0–15 and 16–31. The direct latency/initiation interval is 7/1 for CT butterflies and ML-DSA GS; it is 8/2 for ML-KEM GS and scalar products. The extra reduction stage also changes the ML-KEM baseline, so application cycles from the previous K-only design cannot be attributed to this shared datapath. Registering pair-selected operands before the multiplier separates lane routing from the DSP path without adding interface latency.

TABLE I
PRIMARY EXPERIMENTAL CONFIGURATION.

Processor	RV32IM, 1 core, F/D disabled
Warp / execution width	8 resident warps, 32 lanes
Memory	16 KiB each: I-cache, D-cache, local
Outer caches	L2 and L3 disabled
Workloads	KEM-768 / DSA-65, each $M = 1, 8$
Integration timing	XRT RTL simulation
Routed cores	K-only / shared K/D; V80, 250 MHz

The integration preserves the ML-KEM library API [15]. Lane zero executes scalar control, broadcasts primitive arguments, and activates the warp for cooperative work; the wrapper then restores leader execution. One-shot SHAKE retains its state in registers across absorb, permutation, and squeeze. Incremental SHAKE stores the context between API calls. Cooperative polynomial arithmetic and a warp-level allocation arena avoid replicating data and allocations across 32 lanes. This mapping allows the Keccak backend to change while keeping NTT, polynomial work, and request semantics fixed. ML-DSA uses the same Keccak units through its permutation hook. Its NTT hooks execute three lane-local and five paired layers; the inverse applies the hardware Montgomery scale. The NTT comparison fixes other arithmetic. Separately, W32 lanes reuse NTTMUL.D for independent pointwise products. Five-term dot products retain one 64-bit sum and final reduction per coefficient in software.

IV. IMPLEMENTATION AND EVALUATION

A. Experimental setup

Table I summarizes the Vortex configuration. XRT integration simulation exercises the accelerator interface; SimX supplies controlled timing experiments. The six-backend ML-KEM comparison fixes one all-enabled K-only RTL build and application structure. Every arm passes byte-exact checks of public/secret keys, ciphertext, and both shared secrets against the reference. SimX/XRT retire identical instructions and differ by at most 2.71% in whole-launch cycles across these six backends.

Figure 5 covers the complete device launch, excluding host transfers. Barriers isolate permutation measurements from other warps’ setup and output traffic. Physical results are independent Vivado 2025.1 OPT3 out-of-context core implementations, one run per variant. They measure post-route cost and timing, not board execution or energy.

B. Complete-request acceleration

Figure 5 fixes the NTT and polynomial path while varying Keccak. C and PQRV are scalar C and optimized RV32 assembly; Shuffle shares Stage/Round’s 25-lane placement and is their direct software baseline.

Stage provides $1.462 \times / 1.552 \times$ speedup over Shuffle at one/eight requests. Round reaches 5.20/12.28 million cycles, or $1.520 \times / 1.583 \times$ over Shuffle. Its larger $4.742 \times / 3.074 \times$ gains over PQRV also include software state distribution.

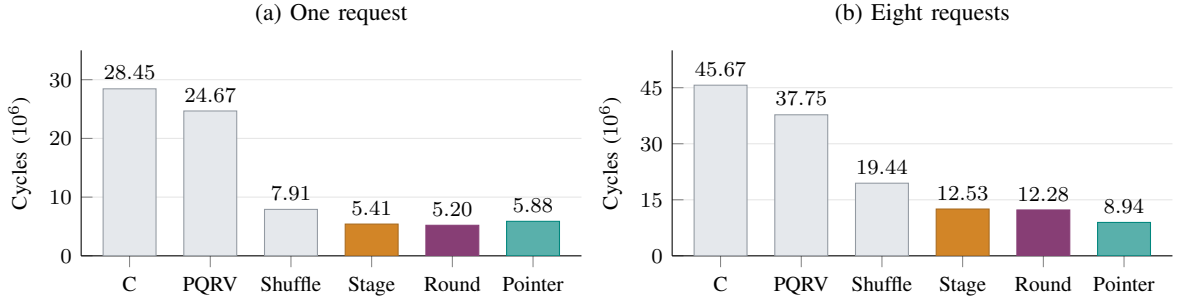


Fig. 5. Complete ML-KEM-768 round-trip cycles on one common XRT RTL configuration; lower is better. Each batch contains M independent requests. NTT and the remaining polynomial path are identical in all arms. Stage uses its looped routine; the separate equal-expansion control in Fig. 7 isolates round fusion.

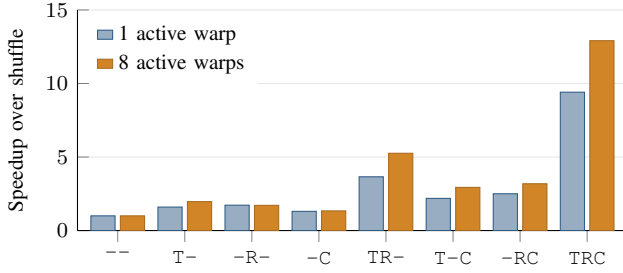


Fig. 6. Eight-way instruction-use ablation in a separate processor RTL experiment. T/R/C select θ , $\rho\pi$, and $\chi\iota$ collectives; a dash retains that software stage. Speedup uses the shuffle-only arm at the same active-warp count and the marginal cost defined in the text.

The preferred backend changes with concurrency. Round is $1.130\times$ faster than Pointer at one request, whereas Pointer is $1.374\times$ faster than Round at eight. This establishes a crossover at the tested request counts, without isolating one arbitration or cache mechanism as its cause.

C. Which stage operations matter

Figure 6 tests all eight software/collective combinations on the same Stage-enabled W8T32 processor RTL. This varies instruction use, not area. It removes fixed interval overhead using marginal cost $[\text{span}(8) - \text{span}(4)]/(4W)$, where W is the number of active warps and each warp holds one state.

Using all stages improves the dependent one-warp cost by $9.408\times$ and eight-warp throughput by $12.909\times$. The best individual replacement changes with load: $\rho\pi$ wins at one warp and θ at eight. No single stage captures the full benefit.

D. What additional round fusion buys

Figure 7 fixes mapping and loop expansion: each active warp holds one state and performs 64 dependent permutations. Against expanded Stage, Round still provides $3.168\times/3.291\times$ permutation throughput at one/eight warps. This isolates fusion from loop overhead; scalar software with multiple states per warp has a different occupancy tradeoff.

In the separate matched SimX application control, both routines are expanded: Stage/Round take 5.342/5.198 million cycles at $M = 1$ and 12.336/11.938 million at $M = 8$. The

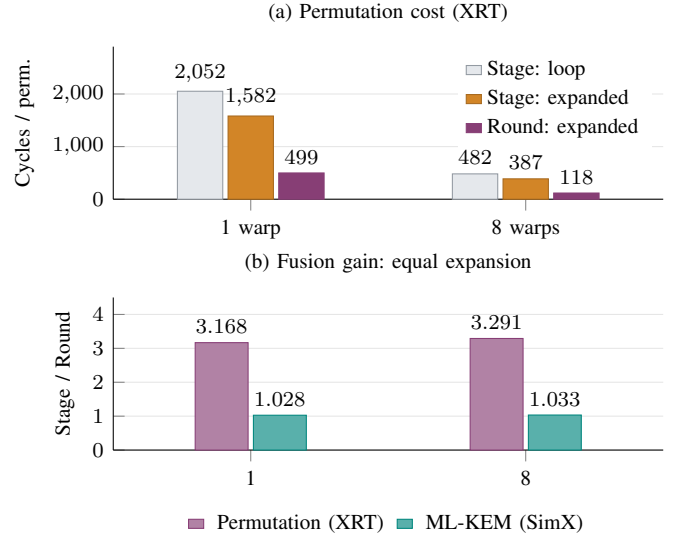


Fig. 7. Keccak granularity with 64 dependent permutations per state. (a) XRT cycles per permutation, one state per active warp. (b) Equal-expansion Stage/Round gain: XRT permutation versus matched SimX complete KEM at one/eight active warps or requests.

complete-request gains are only $1.028\times/1.033\times$ because little permutation work remains after Stage acceleration.

Independent phase probes explain the saturation: permutation occupies 4.964% of the instrumented Stage request and 1.266% for Round, while absorb and squeeze together still occupy 11.661% and 12.902%. The matched probe overhead is 6.6–8.3%, so these fractions locate residual work without predicting an exact uninstrumented speedup.

E. One NTT unit for two schemes

Pointer Keccak and other polynomial work are fixed in one XRT build. Figure 8 compares each scheme's W32 software NTT with the shared instructions using the sum of keypair, operation, and final on-device intervals. The hardware saves 4.913% of complete KEM cycles and 5.939% of complete DSA cycles; scalar-to-W32 mapping gains are excluded. These requests execute 15/9 ML-KEM and 46/49 ML-DSA forward/inverse NTTs, respectively.

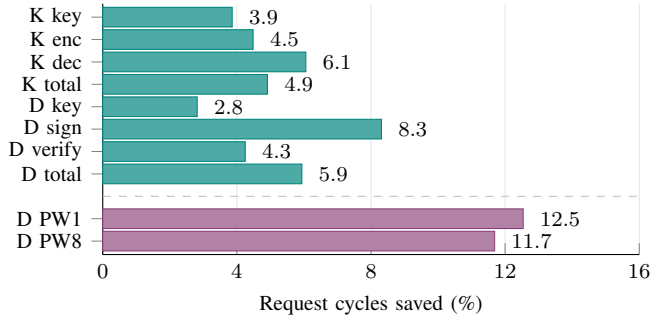


Fig. 8. XRT cycle reductions with Pointer Keccak fixed. NTT bars compare shared instructions with W32 register-shuffle software. PW bars separately compare DSA pointwise mappings with scalar C at $M = 1, 8$; the latter uses batch makespan. K: ML-KEM-768; D: ML-DSA-65. Gains are not additive.

TABLE II
FINAL SHARED-CORE M1 PHASE SHARES (% , XRT).

Scheme	Perm.	NTT/I	Products	Sponge	Rest
KEM	0.516	2.114	2.395	26.785	68.190
DSA	0.334	3.251	2.417	–	93.997

Both NTT paths pass byte-exact XRT request checks, with identical SimX/XRT instruction counts and launch-cycle gaps below 0.879%. Separately, `NTTMUL.D` pointwise products and W32 five-term sums reduce matched XRT DSA cycles by 12.538% at $M = 1$ and batch makespan from 77.071 to 68.064 million cycles (11.687%) at $M = 8$. No RTL is added. Eight distinct inputs exercise 2–11 signing attempts, inferred from L5 call counts. All 32 runs match portable-C keys and signatures; SimX/XRT retire identical instructions and differ by at most 0.593% in launch cycles or makespan. Per-request intervals overlap and are not summed for throughput. The M1/M8 comparisons use separately matched binaries; the pointwise and NTT gains are not additive.

Table II profiles the final shared core with Pointer Keccak. Products denotes KEM mulcache/basemul or DSA ordinary/L5 pointwise; Sponge denotes absorb/squeeze. Rest includes KEM reduction and all unmeasured work; DSA absorb/squeeze remain in Rest. Probe overhead is 6.398%/0.666% against matched uninstrumented KEM/DSA requests. The small measured NTT/product fractions motivate testing further resource sharing, but do not predict its M8 throughput or identify the dominant part of DSA’s residual.

F. NTT communication and resource selection

Holding narrow multiplication fixed, paired butterflies save 2.56%/8.23% of matched SimX KEM intervals at one/eight requests over register shuffles. A stronger shared-memory baseline groups layers and transposes coefficients. Matched XRT intervals are 7.792/11.049 million cycles for that baseline, versus 7.711/10.523 million for full-bank register collectives. These earlier pointer-enabled runs include floating-point hardware; comparisons remain within that family.

TABLE III
INDEPENDENT 250-MHz RV32IM POST-ROUTE CORES. K ROWS USE THE 16-BIT NTT; SHARED ROWS USE ONE K/D UNIT. F/D ARE OFF; ALL USE 133 BRAM TILES AND ROUTE WITHOUT ERRORS.

Variant	LUTs	FFs	DSPs	WNS (ns)
K-only NTT	220,950	131,798	112	+0.004
K + Stage	223,444	133,512	112	+0.004
K + Round	222,935	138,065	112	+0.007
Shared K/D	230,739	137,252	176	+0.018
Shared + Stage	235,887	139,478	176	+0.018

Reducing the K-only bank from 32 to 16 multipliers changes matched KEM intervals by at most 0.041% while saving 14,767 LUTs, 1,565 flip-flops, and 16 DSPs at 250 MHz. This half-bank is the starting point for the shared 32-bit datapath, whose physical cost is measured separately.

G. Routed cost and RV64 sensitivity

Table III reports five independent RV32IM post-route cores, all closing 250 MHz without routing errors. Relative to K-only NTT, Stage adds 1.129% LUTs/1.300% flip-flops and Round adds 0.898%/4.755%. Sharing K/D adds 4.430% LUTs, 4.138% flip-flops, and 64 DSPs over K-only NTT. Adding Stage to the shared core costs 2.231% LUTs/1.622% flip-flops and no DSPs. The all-enabled XRT timing build is separate from these area builds.

RV64 halves Keccak collective issues, yet complete-request instructions fall only 7.00% (Stage) and 5.73% (Round). Matched XRT one-request cycles improve 3.68%/2.54%; at eight requests, Stage gains 0.40% and Round loses 0.18%. RV64 NTT-only uses 429,067 LUTs versus RV32’s 220,950, while its Stage/Round WNS is $-0.148/-0.013$ ns at 250 MHz. The wider core does not improve this measured cost-performance point.

V. CONCLUSION

Register-resident Keccak collectives and one shared K/D NTT unit accelerate byte-exact ML-KEM and ML-DSA requests. Fused rounds improve isolated permutations far more than complete requests; the pointer engine wins at higher measured concurrency. Reusing the NTT multiplier bank for ML-DSA pointwise work and distributing L5 sums across lanes needs no additional hardware and improves both single and eight-request XRT execution.

Content generated by AI. AI tools assisted with drafting, literature organization, and figure code. Experimental results come from archived implementation measurements.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] —, “Module-lattice-based digital signature standard,” NIST, Tech. Rep. FIPS 204, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [3] J. Zhang, Y. Yan, J. Huang, and Ç. K. Koç, “Optimized software implementation of Keccak, Kyber, and Dilithium on RV{32,64}IM{B}{V},” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 632–655, Dec. 2024.
- [4] T. Zhou, F. Zheng, Z. Xie, W. Tang, G. Fan, Y. Ning, Y. Bian, J. Lin, and J. Jing, “ML-Cube: Accelerating module-lattice-based cryptography using machine learning accelerators with a memory-less design,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2025, pp. 1829–1843.
- [5] M. B. Sullivan, S. Ghosh, D. R. Johnson, G. Suh, C. Joseph, and M. Y. Siu, “Warp-collective instructions to accelerate cryptographic primitives,” U.S. patent application publication US 2026/0222190 A1, Jul. 2026, applicant: NVIDIA Corp.; published July 30, 2026, Listing 3. [Online]. Available: <https://patents.justia.com/patent/20260222190>
- [6] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3d-graphics research,” in *Proc. 54th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 754–766.
- [7] National Institute of Standards and Technology, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Tech. Rep. FIPS 202, 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [8] J. Park, S. Lee, and J. Lee, “NTT-PIM: Row-centric architecture and mapping for efficient number-theoretic transform on PIM,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [9] J. Zhang, M. Imani, and E. Sadredini, “BP-NTT: Fast and compact in-SRAM number theoretic transform with bit-parallel modular multiplication,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [10] X. Ren, Z. Chen, Z. Gu, Y. Lu, R. Zhong, W. jie Lu, J. Zhang, Y. Zhang, H. Wu, X. Zheng, H. Liu, T. Chu, C. Hong, C. Wei, D. Niu, and Y. Xie, “CHAM: A customized homomorphic encryption accelerator for fast matrix-vector product,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [11] R. Yin, Y. Li, and P. Panda, “PacQ: A SIMT microarchitecture for efficient dataflow in hyper-asymmetric GEMMs,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [12] Y. Yao, C. Zhang, C. Qi, R. Chen, J. Wang, Z. Fu, N. Jing, X. Liang, and Z. Song, “SynGPU: Synergizing CUDA and bit-serial tensor cores for vision transformer acceleration on GPU,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [13] T. Fritzmann, G. Sigl, and J. Sepúlveda, “RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 4, pp. 239–280, 2020.
- [14] P. Nannipieri, S. D. Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, “A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms,” *IEEE Access*, vol. 9, pp. 150 798–150 808, 2021.
- [15] PQ Code Package contributors, “mlkem-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlkem-native>